

Laporan Pemrograman Backend

Nauval Januarta Putra Aryanto

C4/434231039

Github : <https://github.com/nauvaljanuarta/alumni-be.git>

Models

Alumni

```
type Alumni struct {
    ID          primitive.ObjectID `bson:"_id,omitempty" json:"id,omitempty"`
    NIM         string              `bson:"nim" json:"nim"`
    Nama        string              `bson:"nama" json:"nama"`
    Jurusan     string              `bson:"jurusan" json:"jurusan"`
    Angkatan    int                 `bson:"angkatan" json:"angkatan"`
    TahunLulus  int                 `bson:"tahun_lulus" json:"tahun_lulus"`
    Email       string              `bson:"email" json:"email"`
    NoTelepon   string              `bson:"no_telepon" json:"no_telepon"`
    Alamat      string              `bson:"alamat" json:"alamat"`
    Fakultas    string              `bson:"fakultas" json:"fakultas"`
    Role        string              `bson:"role" json:"role"`
    Password    string              `bson:"password,omitempty" json:"- "` //
    disembunyikan saat response
    CreatedAt   time.Time           `bson:"created_at" json:"created_at"`
    UpdatedAt   time.Time           `bson:"updated_at" json:"updated_at"`
}
```

```
type CreateAlumni struct {
    NIM         string `bson:"nim" json:"nim"`
    Nama        string `bson:"nama" json:"nama"`
    Jurusan     string `bson:"jurusan" json:"jurusan"`
    Angkatan    int    `bson:"angkatan" json:"angkatan"`
    TahunLulus  int    `bson:"tahun_lulus" json:"tahun_lulus"`
    Email       string `bson:"email" json:"email"`
    NoTelepon   string `bson:"no_telepon" json:"no_telepon"`
    Alamat      string `bson:"alamat" json:"alamat"`
    Fakultas    string `bson:"fakultas" json:"fakultas"`
    Role        string `bson:"role" json:"role"`
    Password    string `bson:"password" json:"password"`
}
```

```
type UpdateAlumni struct {
    Nama        string `bson:"nama,omitempty" json:"nama"`
    Jurusan     string `bson:"jurusan,omitempty" json:"jurusan"`
    Angkatan    int    `bson:"angkatan,omitempty" json:"angkatan"`
}
```

```

    TahunLulus int    `bson:"tahun_lulus,omitempty" json:"tahun_lulus"`
    Email      string `bson:"email,omitempty" json:"email"`
    NoTelepon  string `bson:"no_telepon,omitempty" json:"no_telepon"`
    Alamat     string `bson:"alamat,omitempty" json:"alamat"`
    Fakultas  string `bson:"fakultas,omitempty" json:"fakultas"`
    Role       string `bson:"role,omitempty" json:"role"`
    Password   string `bson:"password,omitempty" json:"password"`
}

type LoginRequest struct {
    Email      string `json:"email"`
    Password   string `json:"password"`
}

type LoginResponse struct {
    Alumni Alumni `json:"alumni"`
    Token  string  `json:"token"`
}

type JWTClaims struct {
    UserID primitive.ObjectID `json:"user_id"`
    Email  string                  `json:"email"`
    Role   string                  `json:"role"`
    jwt.RegisteredClaims
}

```

Pekerjaan

```

type Pekerjaan struct {
    ID primitive.ObjectID `bson:"_id,omitempty"
json:"id,omitempty"`
    AlumniID primitive.ObjectID `bson:"alumni_id" json:"alumni_id"`
    NamaPerusahaan string      `bson:"nama_perusahaan"
json:"nama_perusahaan"`
    PosisiJabatan string      `bson:"posisi_jabatan"
json:"posisi_jabatan"`
    BidangIndustri string      `bson:"bidang_industri"
json:"bidang_industri"`
    LokasiKerja string      `bson:"lokasi_kerja"
json:"lokasi_kerja"`
    GajiRange *string `bson:"gaji_range,omitempty"
json:"gaji_range,omitempty"`
    TanggalMulaiKerja time.Time `bson:"tanggal_mulai_kerja"
json:"tanggal_mulai_kerja"`
    TanggalSelesaiKerja
*time.Time `bson:"tanggal_selesai_kerja,omitempty"
json:"tanggal_selesai_kerja,omitempty"`
}

```

```

    StatusPekerjaan    string           `bson:"status_pekerjaan"
json:"status_pekerjaan"`
    DeskripsiPekerjaan *string           `bson:"deskripsi_pekerjaan,omitempty"
json:"deskripsi_pekerjaan,omitempty"`
    CreatedAt          time.Time         `bson:"created_at" json:"created_at"`
    UpdatedAt          time.Time         `bson:"updated_at" json:"updated_at"`
    IsDeleted           bool            `bson:"is_deleted" json:"is_deleted"`
}

type CreatePekerjaan struct {
    AlumniID            string           `bson:"alumni_id" json:"alumni_id"`
    NamaPerusahaan      string           `bson:"nama_perusahaan"
json:"nama_perusahaan"`
    PosisiJabatan       string           `bson:"posisi_jabatan" json:"posisi_jabatan"`
    BidangIndustri       string           `bson:"bidang_industri"
json:"bidang_industri"`
    LokasiKerja         string           `bson:"lokasi_kerja" json:"lokasi_kerja"`
    GajiRange            *string          `bson:"gaji_range,omitempty"
json:"gaji_range,omitempty"`
    TanggalMulaiKerja    string           `bson:"tanggal_mulai_kerja"
json:"tanggal_mulai_kerja"`
    TanggalSelesaiKerja *string          `bson:"tanggal_selesai_kerja,omitempty"
json:"tanggal_selesai_kerja,omitempty"`
    StatusPekerjaan      string           `bson:"status_pekerjaan"
json:"status_pekerjaan"`
    DeskripsiPekerjaan   *string          `bson:"deskripsi_pekerjaan,omitempty"
json:"deskripsi_pekerjaan,omitempty"`
    IsDeleted            bool            `bson:"is_deleted" json:"is_deleted"`
    CreatedAt            time.Time       `bson:"created_at" json:"created_at"`
    UpdatedAt            time.Time       `bson:"updated_at" json:"updated_at"`
}

type UpdatePekerjaan struct {
    NamaPerusahaan      string           `bson:"nama_perusahaan,omitempty"
json:"nama_perusahaan,omitempty"`
    PosisiJabatan       string           `bson:"posisi_jabatan,omitempty"
json:"posisi_jabatan,omitempty"`
    BidangIndustri       string           `bson:"bidang_industri,omitempty"
json:"bidang_industri,omitempty"`
    LokasiKerja         string           `bson:"lokasi_kerja,omitempty"
json:"lokasi_kerja,omitempty"`
    GajiRange            *string          `bson:"gaji_range,omitempty"
json:"gaji_range,omitempty"`
    TanggalMulaiKerja    string           `bson:"tanggal_mulai_kerja,omitempty"
json:"tanggal_mulai_kerja,omitempty"`
    TanggalSelesaiKerja *string          `bson:"tanggal_selesai_kerja,omitempty"
json:"tanggal_selesai_kerja,omitempty"`
}

```

```

    StatusPekerjaan      string      `bson:"status_pekerjaan,omitempty"
    json:"status_pekerjaan,omitempty"
    DeskripsiPekerjaan   *string    `bson:"deskripsi_pekerjaan,omitempty"
    json:"deskripsi_pekerjaan,omitempty"
    UpdatedAt            time.Time   `bson:"updated_at,omitempty"
    json:"updated_at,omitempty"
    IsDeleted            bool        `bson:"is_deleted" json:"is_deleted"
    CreatedAt            time.Time   `bson:"created_at" json:"created_at"
}

```

Repository

Alumni

```

/ interface untuk kontrak repository alumni
type IAlumniRepository interface {
    GetAlumni(ctx context.Context, search string, limit, offset int, sortBy,
order string) ([]models.Alumni, error)
    GetAlumniByID(ctx context.Context, id string) (*models.Alumni, error)
    GetByEmail(ctx context.Context, email string) (*models.Alumni, error)
    CreateAlumni(ctx context.Context, req *models.Alumni) (*models.Alumni,
error)
    UpdateAlumni(ctx context.Context, id string, req *models.Alumni) error
    DeleteAlumni(ctx context.Context, id string) error
    Count(ctx context.Context, search string) (int, error)
}

type AlumniRepository struct {
    collection *mongo.Collection
}

func NewAlumniRepository(db *mongo.Database) IAlumniRepository {
    return &AlumniRepository{
        collection: db.Collection("alumni"),
    }
}

func (r *AlumniRepository) Count(ctx context.Context, search string) (int,
error) {
    filter := bson.M{}
    if search != "" {
        filter = bson.M{
            "$or": []bson.M{
                {"nama": bson.M{"$regex": search, "$options": "i"}},
                {"nim": bson.M{"$regex": search, "$options": "i"}},
                {"email": bson.M{"$regex": search, "$options": "i"}},
                {"jurusan": bson.M{"$regex": search, "$options": "i"}},
                {"fakultas": bson.M{"$regex": search, "$options": "i"}},
            },
        }
    }
}

```

```

    },
    }
}

count, err := r.collection.CountDocuments(ctx, filter)
if err != nil {
    return 0, err
}

return int(count), nil
}

func (r *AlumniRepository) GetAlumni(ctx context.Context, search string,
limit, offset int, sortBy, order string) ([]models.Alumni, error) {
    filter := bson.M{}
    if search != "" {
        filter = bson.M{
            "$or": []bson.M{
                {"nama": bson.M{"$regex": search, "$options": "i"}},
                {"nim": bson.M{"$regex": search, "$options": "i"}},
                {"email": bson.M{"$regex": search, "$options": "i"}},
                {"jurusan": bson.M{"$regex": search, "$options": "i"}},
                {"fakultas": bson.M{"$regex": search, "$options": "i"}},
            },
        }
    }
}

findOptions := mongoOptions(limit, offset, sortBy, order)

cursor, err := r.collection.Find(ctx, filter, findOptions)
if err != nil {
    return nil, err
}
defer cursor.Close(ctx)

var alumniList []models.Alumni
if err := cursor.All(ctx, &alumniList); err != nil {
    return nil, err
}

return alumniList, nil
}

func mongoOptions(limit, offset int, sortBy, order string)
*options.FindOptions {
    opts := &options.FindOptions{}
    if limit > 0 {
        opts.SetLimit(int64(limit))
        opts.SetSkip(int64(offset))
    }
}

```

```

    }
    if sortBy != "" {
        sortOrder := 1
        if order == "desc" {
            sortOrder = -1
        }
        opts.SetSort(bson.D{{Key: sortBy, Value: sortOrder}})
    }
    return opts
}

func (r *AlumniRepository) GetByEmail(ctx context.Context, email string)
(*models.Alumni, error) {
    var alumni models.Alumni
    filter := bson.M{"email": email}

    err := r.collection.FindOne(ctx, filter).Decode(&alumni)
    if err != nil {
        if err == mongo.ErrNoDocuments {
            return nil, nil
        }
        return nil, err
    }

    return &alumni, nil
}

func (r *AlumniRepository) GetAlumniByID(ctx context.Context, id string)
(*models.Alumni, error) {
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return nil, err
    }

    var alumni models.Alumni
    err = r.collection.FindOne(ctx, bson.M{"_id": objID}).Decode(&alumni)
    if err != nil {
        if err == mongo.ErrNoDocuments {
            return nil, nil
        }
        return nil, err
    }

    return &alumni, nil
}

func (r *AlumniRepository) CreateAlumni(ctx context.Context, req
*models.Alumni) (*models.Alumni, error) {
    req.ID = primitive.NilObjectID

```

```

req.CreatedAt = time.Now()
req.UpdatedAt = time.Now()

result, err := r.collection.InsertOne(ctx, req)
if err != nil {
    return nil, err
}

req.ID = result.InsertedID.(primitive.ObjectID)
return req, nil
}

func (r *AlumniRepository) UpdateAlumni(ctx context.Context, id string, req
*models.Alumni) error {
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return err
    }

    req.UpdatedAt = time.Now()
    update := bson.M{"$set": req}

    _, err = r.collection.UpdateOne(ctx, bson.M{"_id": objID}, update)
    return err
}

func (r *AlumniRepository) DeleteAlumni(ctx context.Context, id string) error
{
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return err
    }

    _, err = r.collection.DeleteOne(ctx, bson.M{"_id": objID})
    return err
}

```

Pekerjaan

```

type IPekerjaanRepository interface {
    GetAll(ctx context.Context, search, sortBy, order string, limit, offset int)
([]models.Pekerjaan, error)
    Count(ctx context.Context, search string) (int64, error)
    GetByID(ctx context.Context, id primitive.ObjectID) (*models.Pekerjaan,
error)
    GetByAlumniID(ctx context.Context, alumniID primitive.ObjectID)
([]models.Pekerjaan, error)
}

```

```

    Create(ctx context.Context, req models.CreatePekerjaan)
(*mongo.InsertOneResult, error)
    Update(ctx context.Context, id primitive.ObjectID, req
models.UpdatePekerjaan) error
    Delete(ctx context.Context, id primitive.ObjectID) error
    SoftDelete(ctx context.Context, id primitive.ObjectID) error
    SoftDeleteBulk(ctx context.Context) error
    Restore(ctx context.Context, id primitive.ObjectID) error
    GetTrash(ctx context.Context, search, sortBy, order string, limit, offset
int) ([]models.Pekerjaan, error)
    CountTrash(ctx context.Context, search string) (int64, error)
    DeleteTrash(ctx context.Context, id primitive.ObjectID) error
}

type pekerjaanRepository struct {
    collection *mongo.Collection
}

func NewPekerjaanRepository(db *mongo.Database) IPekerjaanRepository {
    return &pekerjaanRepository{
        collection: db.Collection("pekerjaan_alumni"),
    }
}

func (r *pekerjaanRepository) GetAll(ctx context.Context, search, sortBy,
order string, limit, offset int) ([]models.Pekerjaan, error) {
    filter := bson.M{"is_deleted": false}

    if search != "" {
        filter["$or"] = []bson.M{
            {"nama_perusahaan": bson.M{"$regex": search, "$options": "i"}},
            {"posisi_jabatan": bson.M{"$regex": search, "$options": "i"}},
            {"bidang_industri": bson.M{"$regex": search, "$options": "i"}},
            {"lokasi_kerja": bson.M{"$regex": search, "$options": "i"}},
        }
    }

    sortOrder := 1
    if order == "DESC" || order == "desc" {
        sortOrder = -1
    }

    if sortBy == "" {
        sortBy = "created_at"
    }

    opts := options.Find().
        SetSort(bson.M{sortBy: sortOrder}).
        SetLimit(int64(limit)).

```



```

        SetSkip(int64(offset))

    cursor, err := r.collection.Find(ctx, filter, opts)
    if err != nil {
        return nil, err
    }
    defer cursor.Close(ctx)

    var list []models.Pekerjaan
    if err := cursor.All(ctx, &list); err != nil {
        return nil, err
    }

    return list, nil
}

func (r *pekerjaanRepository) Count(ctx context.Context, search string)
(int64, error) {
    filter := bson.M{"isdeleted": false}

    if search != "" {
        filter["$or"] = []bson.M{
            {"nama_perusahaan": bson.M{"$regex": search, "$options": "i"}},
            {"posisi_jabatan": bson.M{"$regex": search, "$options": "i"}},
            {"bidang_industri": bson.M{"$regex": search, "$options": "i"}},
            {"lokasi_kerja": bson.M{"$regex": search, "$options": "i"}},
        }
    }

    return r.collection.CountDocuments(ctx, filter)
}

func (r *pekerjaanRepository) GetByID(ctx context.Context, id
primitive.ObjectID) (*models.Pekerjaan, error) {
    var pekerjaan models.Pekerjaan
    filter := bson.M{"_id": id}

    err := r.collection.FindOne(ctx, filter).Decode(&pekerjaan)
    if err != nil {
        if err == mongo.ErrNoDocuments {
            return nil, nil // Tidak ditemukan
        }
        return nil, err
    }

    return &pekerjaan, nil
}

```

```

func (r *pekerjaanRepository) GetByAlumniID(ctx context.Context, alumniID
primitive.ObjectID) ([]models.Pekerjaan, error) {
    filter := bson.M{
        "alumni_id": alumniID,
        "is_deleted": false,
    }

    cursor, err := r.collection.Find(ctx, filter)
    if err != nil {
        return nil, err
    }
    defer cursor.Close(ctx)

    var list []models.Pekerjaan
    if err := cursor.All(ctx, &list); err != nil {
        return nil, err
    }

    return list, nil
}

func (r *pekerjaanRepository) Create(ctx context.Context, req
models.CreatePekerjaan) (*mongo.InsertOneResult, error) {
    // Convert AlumniID ke ObjectID
    alumniID, err := primitive.ObjectIDFromHex(req.AlumniID)
    if err != nil {
        return nil, errors.New("invalid alumni_id")
    }

    // Parse tanggal mulai
    tMulai, err := time.Parse("2006-01-02", req.TanggalMulaiKerja)
    if err != nil {
        return nil, errors.New("invalid tanggal_mulai_kerja")
    }

    // Parse tanggal selesai jika ada
    var tSelesai *time.Time
    if req.TanggalSelesaiKerja != nil && *req.TanggalSelesaiKerja != "" {
        ts, err := time.Parse("2006-01-02", *req.TanggalSelesaiKerja)
        if err != nil {
            return nil, errors.New("invalid tanggal_selesai_kerja")
        }
        tSelesai = &ts
    }

    now := time.Now()
    doc := bson.M{
        "alumni_id": alumniID,

```

```

        "nama_perusahaan": req>NamaPerusahaan,
        "posisi_jabatan": req.PosisiJabatan,
        "bidang_industri": req.BidangIndustri,
        "lokasi_kerja": req.LokasiKerja,
        "gaji_range": req.GajiRange,
        "tanggal_mulai_kerja": tMulai,
        "tanggal_selesai_kerja": tSelesai,
        "status_pekerjaan": req.StatusPekerjaan,
        "deskripsi_pekerjaan": req.DeskripsiPekerjaan,
        "created_at": now,
        "updated_at": now,
        "is_deleted": false,
    }

    return r.collection.InsertOne(ctx, doc)
}

func (r *pekerjaanRepository) Update(ctx context.Context, id
primitive.ObjectID, req models.UpdatePekerjaan) error {
    filter := bson.M{"_id": id}
    update := bson.M{
        "$set": bson.M{
            "nama_perusahaan": req>NamaPerusahaan,
            "posisi_jabatan": req.PosisiJabatan,
            "bidang_industri": req.BidangIndustri,
            "lokasi_kerja": req.LokasiKerja,
            "gaji_range": req.GajiRange,
            "tanggal_mulai_kerja": req.TanggalMulaiKerja,
            "tanggal_selesai_kerja": req.TanggalSelesaiKerja,
            "status_pekerjaan": req.StatusPekerjaan,
            "deskripsi_pekerjaan": req.DeskripsiPekerjaan,
            "updated_at": time.Now(),
        },
    }

    result, err := r.collection.UpdateOne(ctx, filter, update)
    if err != nil {
        return err
    }

    if result.MatchedCount == 0 {
        return mongo.ErrNoDocuments
    }

    return nil
}

```

```

func (r *pekerjaanRepository) Delete(ctx context.Context, id
primitive.ObjectID) error {
    filter := bson.M{"_id": id}

    result, err := r.collection.DeleteOne(ctx, filter)
    if err != nil {
        return err
    }

    if result.DeletedCount == 0 {
        return mongo.ErrNoDocuments
    }

    return nil
}

```

Connection

```

var DB *mongo.Database
var Client *mongo.Client

func ConnectDB() {
    // dsn := fmt.Sprintf(
    //     "host=%s user=%s password=%s dbname=%s port=%s sslmode=disable",
    //     os.Getenv("DB_HOST"),
    //     os.Getenv("DB_USER"),
    //     os.Getenv("DB_PASS"),
    //     os.Getenv("DB_NAME"),
    //     os.Getenv("DB_PORT"),
    // )

    // var err error
    // DB, err = sql.Open("postgres", dsn)
    // if err != nil {
    //     log.Fatal("Error opening DB:", err)
    // }

    // if err = DB.Ping(); err != nil {
    //     log.Fatal("Error connecting to DB:", err)
    // }

    // var version string
    // if err := DB.QueryRow("SELECT version()").Scan(&version); err != nil {
    //     log.Fatal("Error checking DB version:", err)
    // }

    // log.Println("Connected database :", version)
}

```

```

mongoURI := os.Getenv("MONGO_URI")
if mongoURI == "" {
    mongoURI = "mongodb://localhost:27017"
}

dbName := os.Getenv("DB_NAME")
if dbName == "" {
    dbName = "alumni_db"
}

clientOptions := options.Client().ApplyURI(mongoURI)

ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()

var err error
Client, err = mongo.Connect(ctx, clientOptions)
if err != nil {
    log.Fatal("Error connecting to MongoDB:", err)
}

if err = Client.Ping(ctx, nil); err != nil {
    log.Fatal("Error pinging MongoDB:", err)
}

DB = Client.Database(dbName)

log.Println("Connected to MongoDB database:", dbName)
}

func DBClose() {
    if Client != nil {
        ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
        defer cancel()

        if err := Client.Disconnect(ctx); err != nil {
            log.Println("Error disconnecting from MongoDB:", err)
        } else {
            log.Println("Disconnected from MongoDB")
        }
    }
}
}

```

Route

```
package route

import (
    // "database/sql"
    "pert5/app/repository"
    "pert5/app/service"
    "pert5/middleware"

    "github.com/gofiber/fiber/v2"
    "go.mongodb.org/mongo-driver/mongo"
)

// func SetupRoutes(app *fiber.App, db *sql.DB) {
func SetupRoutes(app *fiber.App, db *mongo.Database) {
    api := app.Group("/pert5")

    // auth login
    alumniRepo := repository.NewAlumniRepository(db)
    authService := service.NewAuthService(alumniRepo)
    api.Post("/login", authService.Login)

    // alumni routes
    alumniService := service.NewAlumniService(alumniRepo)
    alumni := api.Group("/alumni")

    alumni.Get("/", middleware.RequireAuth, alumniService.GetAlumni)
    alumni.Get("/:id", middleware.RequireAuth, alumniService.GetByID)
    alumni.Get("/fakultas/:fakultas", middleware.RequireAuth,
alumniService.GetByFakultas)
    // admin only
    alumni.Post("/", middleware.RequireAuth, middleware.AdminOnly(),
alumniService.Create)
    alumni.Put("/:id", middleware.RequireAuth, middleware.AdminOnly(),
alumniService.Update)
    alumni.Delete("/:id", middleware.RequireAuth, middleware.AdminOnly(),
alumniService.Delete)

    // route pekerjaan
    pekerjaanRepo := repository.NewPekerjaanRepository(db)
    pekerjaanService := service.NewPekerjaanService(pekerjaanRepo)
    pekerjaan := api.Group("/pekerjaan")

    // uts
    pekerjaan.Put("/restore/:id", middleware.RequireAuth,
pekerjaanService.Restore)
    pekerjaan.Get("/trash", middleware.RequireAuth, pekerjaanService.GetTrash)
```

```

    pekerjaan.Delete("/deletetrash/:id", middleware.RequireAuth,
pekerjaanService.DeleteTrash)

// admin & user
pekerjaan.Get("/", middleware.RequireAuth, pekerjaanService.GetAll)
pekerjaan.Get("/:id", middleware.RequireAuth, pekerjaanService.GetByID)
pekerjaan.Put("/softdelete/:id", middleware.RequireAuth,
pekerjaanService.SoftDelete)

// Hanya Admin
pekerjaan.Get("/alumni/:alumni_id", middleware.RequireAuth,
middleware.AdminOnly(), pekerjaanService.GetByAlumniID)
pekerjaan.Post("/", middleware.RequireAuth, middleware.AdminOnly(),
pekerjaanService.Create)
pekerjaan.Put("/:id", middleware.RequireAuth, middleware.AdminOnly(),
pekerjaanService.Update)
pekerjaan.Delete("/:id", middleware.RequireAuth, middleware.AdminOnly(),
pekerjaanService.Delete)
pekerjaan.Put("/softdeletebulk",
middleware.RequireAuth, middleware.AdminOnly(),
pekerjaanService.SoftDeleteBulk)
}

```

Mongo data

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

EXPORT DATA

UPDATE

DELETE

25 1 - 3 of 3

```

{
  "_id": {},
  "nim": "2141720001",
  "nama": "Mauval Januarta",
  "jurusan": "Teknik Informatika",
  "angkatan": 2021,
  "tahun_lulus": 2025,
  "email": "mauval@example.com",
  "no_telepon": "081234567890",
  "alamat": "Jl. Soekarno Hatta No. 45, Malang",
  "fakultas": "Fakultas Teknologi Informasi",
  "role": "user",
  "password": "$2a$10$exampleHashedPassword1234567890",
  "created_at": {},
  "updated_at": {}
}

```

```

{
  "_id": {},
  "nim": "A002",
  "nama": "Pina Andini",
  "jurusan": "Sistem Informasi",
  "angkatan": 2018,
  "tahun_lulus": 2022,
  "email": "pina.andini@example.com",
  "no_telepon": "082134567891",
  "alamat": "Jl. Kenanga No. 7, Surabaya",
  "fakultas": "Teknologi Informasi",
  "role": "user",
  "password": "$2a$14$uXpaKGZggULT5HPA.Aw1QuU1TfAgij6b14WoGHw6KTBHUSLGkwQ3a",
  "created_at": {},
  "updated_at": {}
}

```

```

{
  "_id": {},
  "nim": "434231039",
  "nama": "Mauval Januarta",
  "jurusan": "Teknik Informatika",
  "angkatan": 2019,
  "tahun_lulus": 2023,
  "email": "admingmail.com",
  "no_telepon": "081234567890",
  "alamat": "Jl. Mawar No. 10, Malang",
  "fakultas": "Teknologi Informasi",
  "role": "admin"
}

```

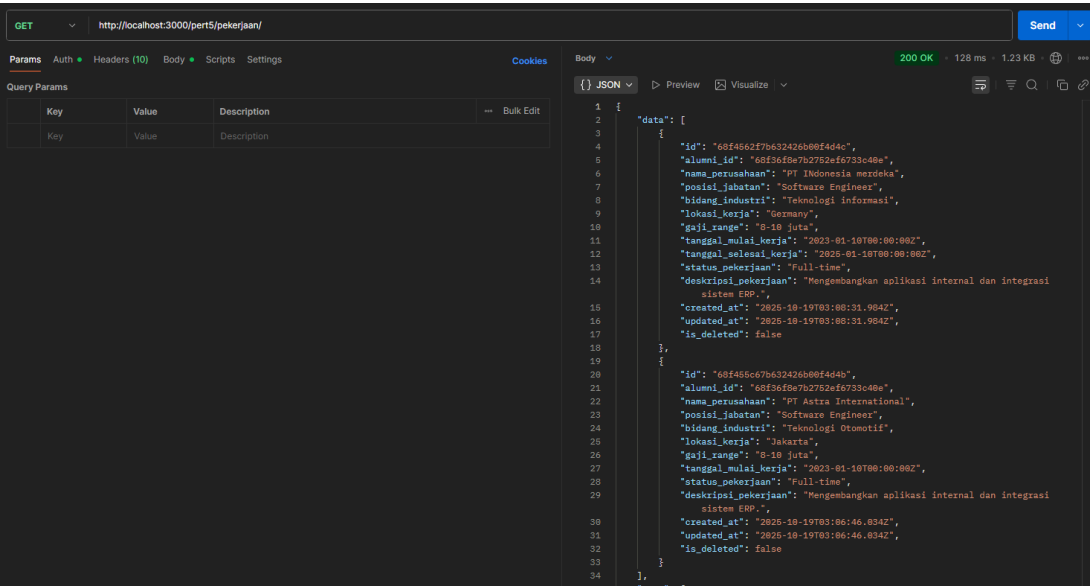
```
ADD DATA      UPDATE      DELETE

_id: ObjectId('68f455c67b632426b00f4db')
posisi_jabatan: "Software Engineer"
bidang_industri: "Teknologi Otomotif"
lokasi_kerja: "Jakarta"
tanggal_mulai_kerja: 2023-01-10T00:00:00.000+00:00
status_pekerjaan: "Full-time"
alumni_id: ObjectId('68f36f8e7b2752ef6733c40e')
nama_perusahaan: "PT Astra International"
gaji_range: "8-10 juta"
tanggal_selesai_kerja: null
deskripsi_pekerjaan: "Mengembangkan aplikasi internal dan integrasi sistem ERP."
created_at: 2025-10-19T03:06:46.034+00:00
updated_at: 2025-10-19T03:06:46.034+00:00
is_deleted: false

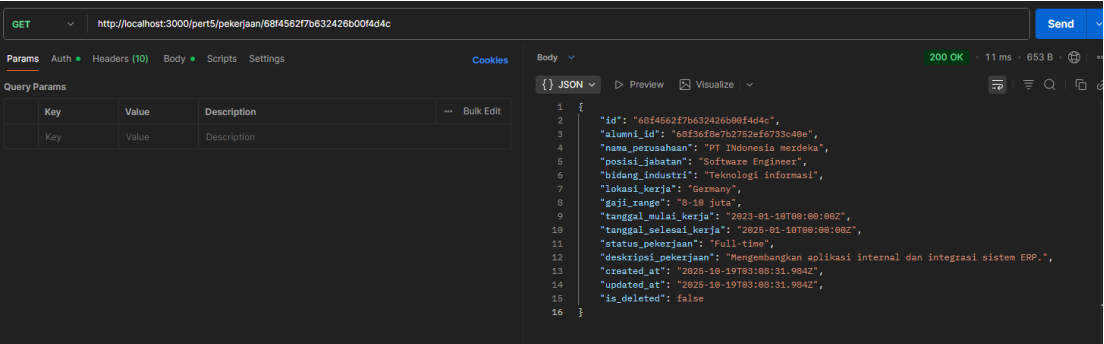
_id: ObjectId('68f45627f7b632426b00f4dc')
posisi_jabatan: "Software Engineer"
lokasi_kerja: "Germany"
gaji_range: "8-10 juta"
tanggal_selesai_kerja: 2025-01-10T00:00:00.000+00:00
status_pekerjaan: "Full-time"
created_at: 2025-10-19T03:08:31.984+00:00
bidang_industri: "Teknologi Informasi"
tanggal_mulai_kerja: 2023-01-10T00:00:00.000+00:00
deskripsi_pekerjaan: "Mengembangkan aplikasi internal dan integrasi sistem ERP."
updated_at: 2025-10-19T03:08:31.984+00:00
is_deleted: false
alumni_id: ObjectId('68f36f8e7b2752ef6733c40e')
nama_perusahaan: "PT Indonesia merdeka"
```


Result

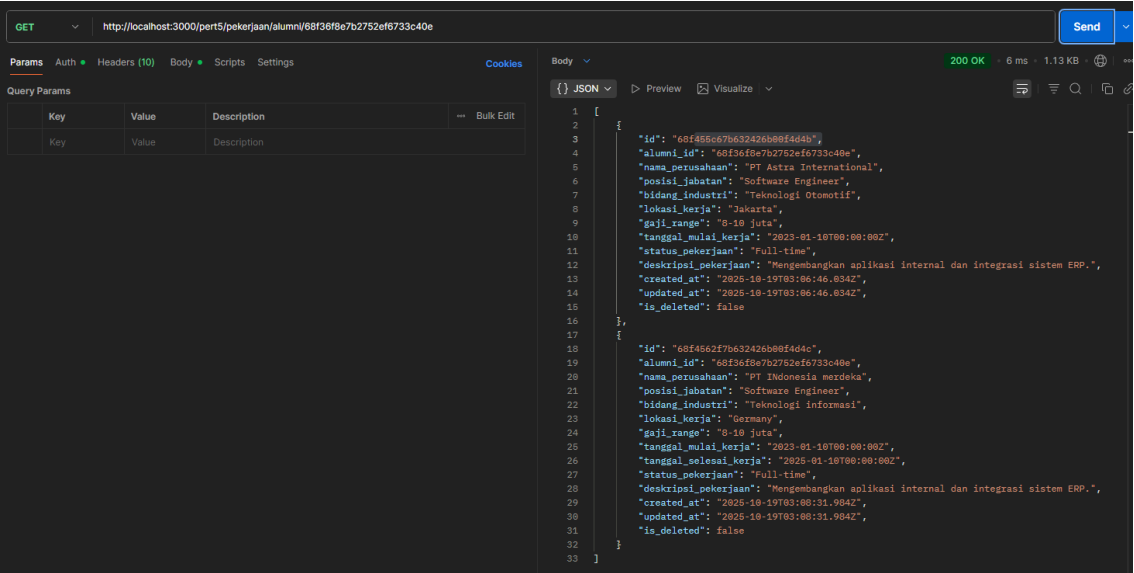
Get all



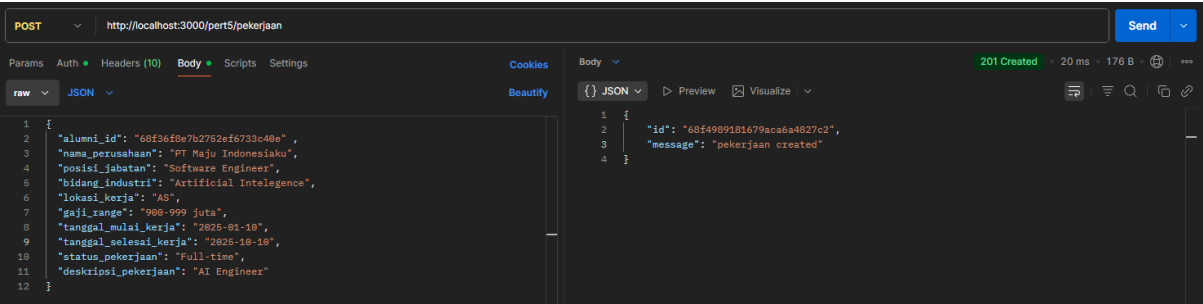
Get pekerjaan by id



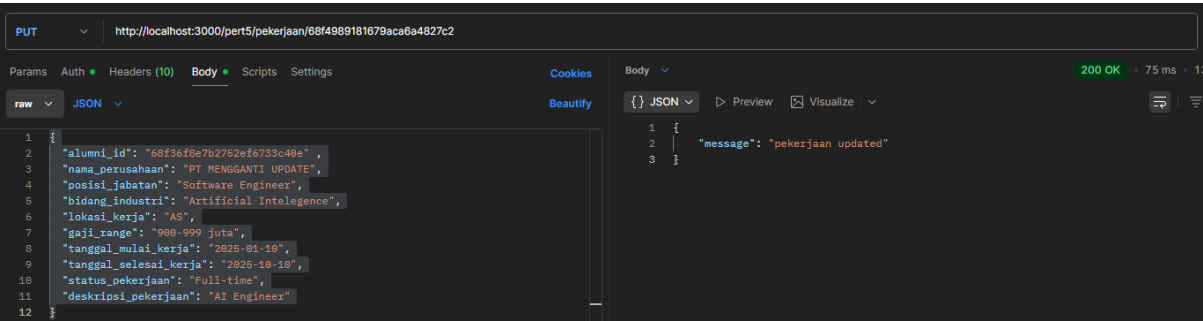
Get pekerjaan by alumni_id



Tambah pekerjaan



Edit pekerjaan



Delete pekerjaan

